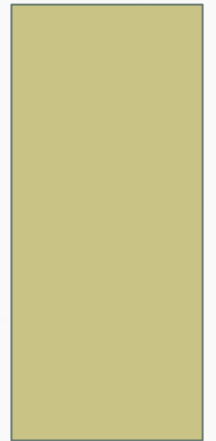


DATA STRUCTURES

EC 200

INTRODUCTION TO LINKED LIST



BOOK READINGS

- Data Abstraction and Problem Solving with C++ by Walls & Mirrors: Chapter 4

PREREQUISITE CONCEPTS

- Arrays
- Dynamic Memory
- Classes and Structures
- Deep copy

DATA STRUCTURES

- In computer science, a **data structure** is a particular way of organizing **data** in a computer so that it can be used efficiently.
 - Array
 - Linked List
 - Stack
 - Queue
 - Trees

DYNAMIC ARRAY APPLICATION

Assume a Student Management System with following features

- Press 1 if you want to enter a record
- Press 2 if you want to delete a record
- Press 3 if you want to search a record
- Press 4 if you want to exit

Which data holder is preferable for such systems ?

STATIC ARRAYS: PROS AND CONS

Major Pros and cons of a static array are

- + Fast element access.
- Impossible to resize.
- Many applications require resizing!
- Required size not always immediately available.

Solution ?

- **Linked List**

LINKED LIST

Linked list

A collection of data in which each element contains the location of the next element.

- Each element contains two parts: **data** and **link**.
- The link contains a pointer (an address) that identifies the next element in the list.

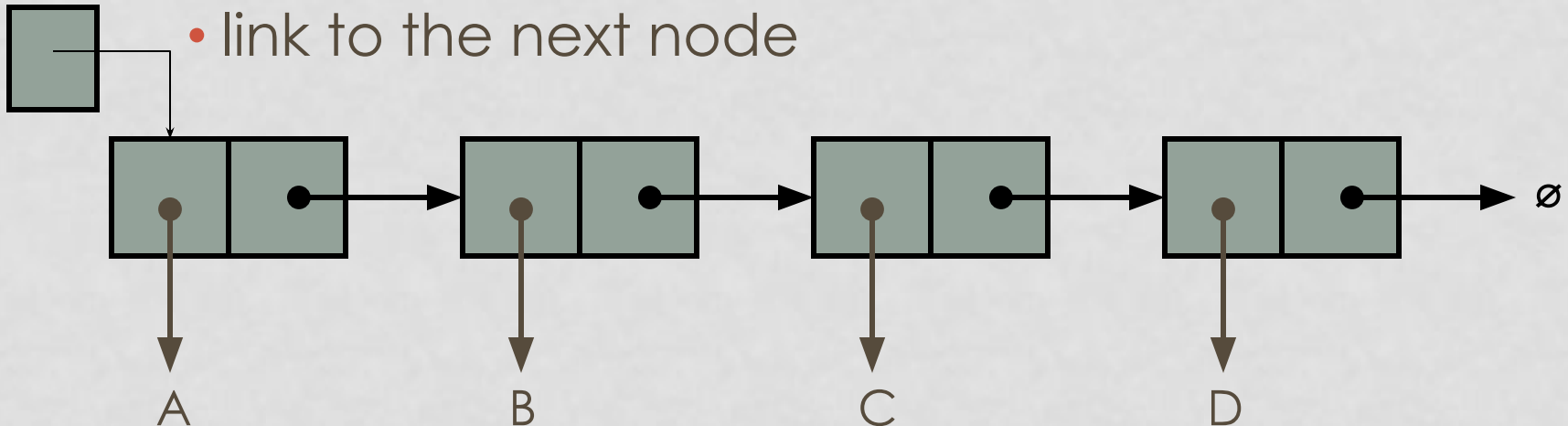
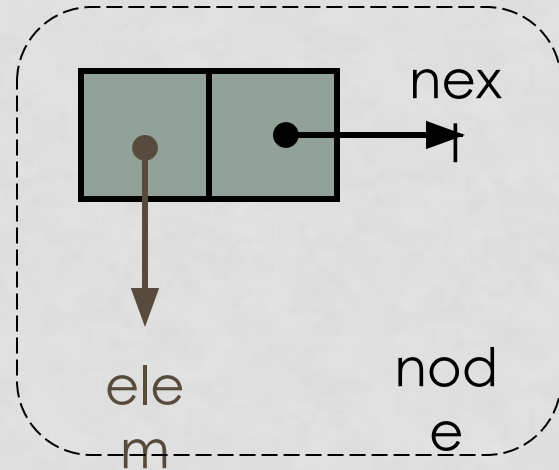
Node



- **Nodes** : the elements in a linked list.
- The nodes in a linked list are called self-referential records.
- Each instance of the record contains a pointer to another instance of the same structural type.

SINGLY LINKED LISTS

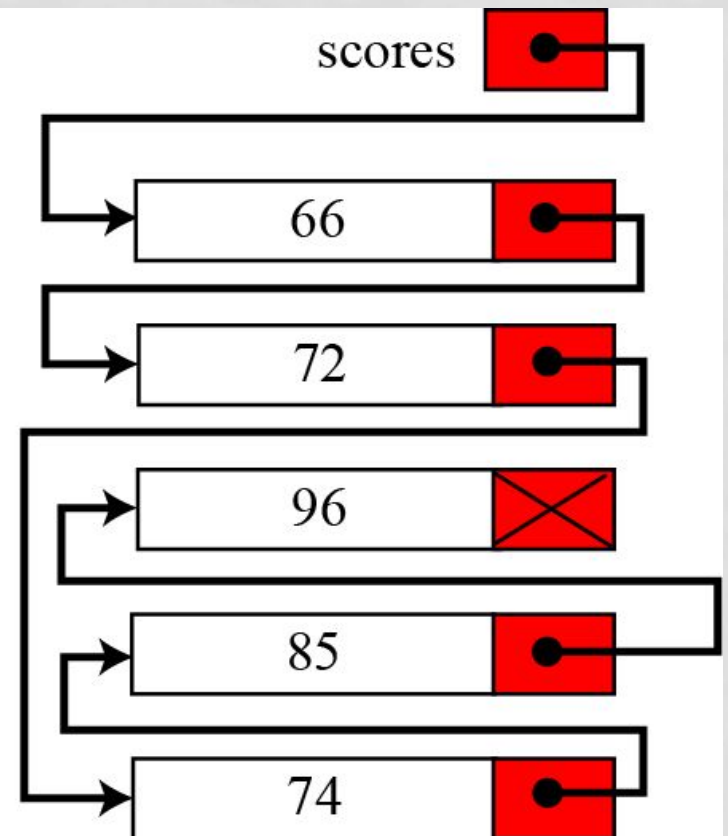
- A singly linked list is a data structure consisting of a sequence of nodes
- Each node stores
 - element
 - link to the next node



MEMORY REPRESENTATION OF LINKED LIST & ARRAY

	scores
scores [1]	66
scores [2]	72
scores [3]	74
scores [4]	85
scores [5]	96

a. Array representation



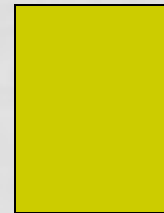
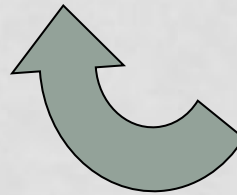
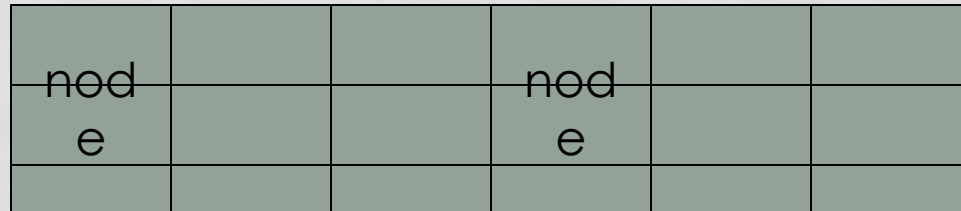
b. Linked list representation

BASIC OPERATIONS IN A SIMPLE LINKED LIST:

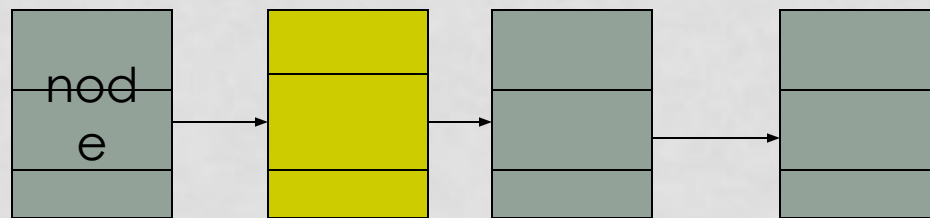
- Insertion
- Deletion
- Searching or Iterating through the list to display items.

ARRAY VS LINKED LIST

Array



Linked List



REPRESENTATION OF NODE IN C++

```
struct Node  
{  
    int data; // data can be of any type  
    Node *next;  
}
```



A SIMPLE LINKED LIST CLASS

- Operations of List
 - Insert / Delete
 - **AtStart**: insert/delete a new node at start
 - **AtEnd**: insert/delete a new node at end
 - **AtAnyPosition**: insert/delete a new node at a particular position
 - **IsEmpty**: determine whether or not the list is empty
 - **FindNode**: find a node with a given value
 - **Traverse**: print all the nodes in the list

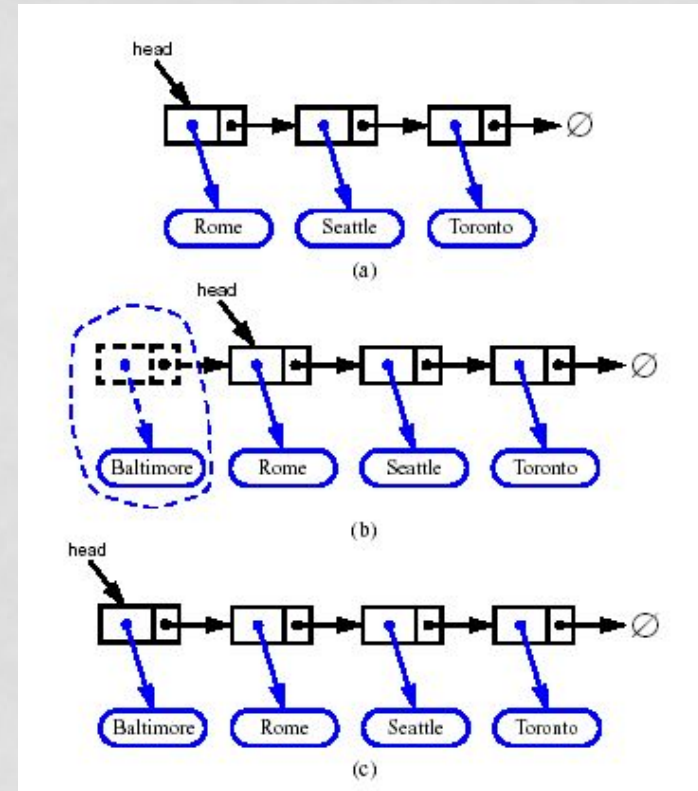
REPRESENTATION OF LINKED LIST IN C++

```
Class linkedList
{
// attributes
Node *head;

public:
// member functions
void insertStart(int v);
void insertEnd(int v);
void deleteStart ();
void deleteEnd();
void traverse();
linkedList();
linkedList(const linkedList& l);
~linkedList();
};
```

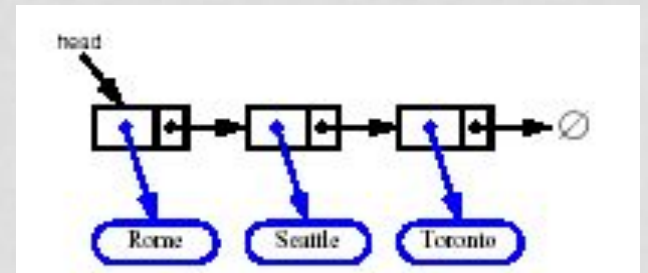
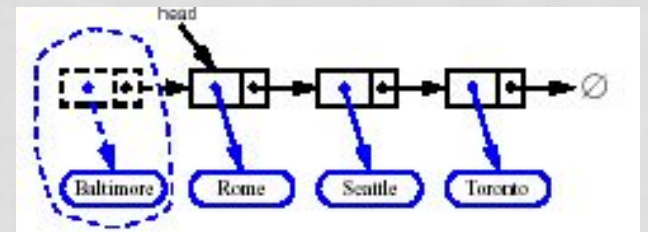
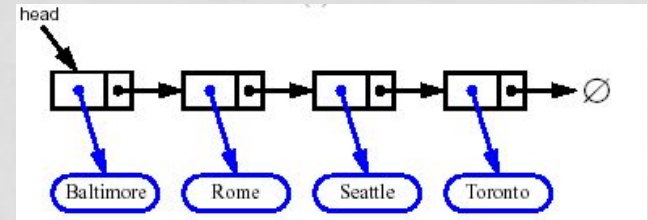
INSERTING AT THE HEAD

1. Allocate a new node
2. Insert new element
3. Make new node point to old head
4. Update head to point to new node



REMOVING AT THE HEAD

1. Hold address of head node in temporary pointer
2. Update head to point to next node in the list
3. Delete the previous head node whose address is in temporary node

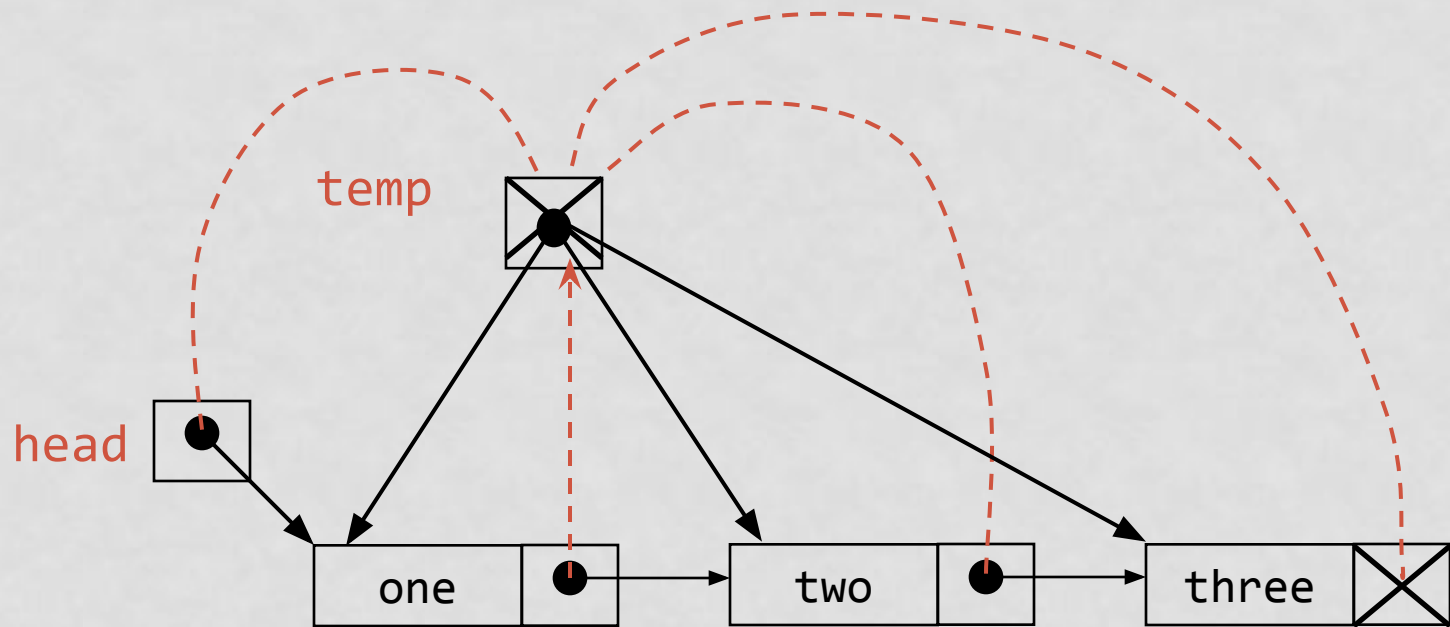


CAUTION!

- Always make sure that your linked list functions work correctly with an empty list.



TRAVERSING A SLL (ANIMATION)



COPY CONSTRUCTOR STEPS

1. Create a temp pointer and assign address of l.head to temp
2. Create a new node n and assign value of temp -> data to n->data
3. Assign address of node created in step 2 to head
4. Move temp to next location
5. Create a pointer temp2 and assign it with head
6. Check temp != NULL
7. Create a new node and assign address to temp2-> next
8. Assign temp-> data to temp2->next -> data
9. Move temp to next node
10. Move temp2 to next node
11. Repeat steps 6 - 9

LINKED LIST APPLICATIONS

- Student Management System
 - Insert a new student In sequence w.r.t Roll no
 - Delete a student anywhere from list
 - Edit a student
- Audio Player
 - Create a play List
 - Play next Track
 - Delete a track from play list

MODIFICATIONS OF LINKED LISTS

- Some modifications of linked lists are
 - Circular Lists
 - Doubly Linked Lists
 - Circular Doubly Linked Lists

SUMMARY

- Linked Lists are more dynamic, as data can be inserted anywhere with less time consumption
- Linked Lists are flexible with respect to size
- Direct access to any node is not possible in Linked List
- Selection of linked list or arrays depends on the requirement of application to be designed and the processing involved on the saved data

ANIMATION SOURCES

- <https://www.youtube.com/watch?v=VmWvpwxa-rM>
- <https://www.youtube.com/watch?v=xiloa2rfAaQ>

TEXT BOOKS

- Chapter 4 Data Abstraction and Problem Solving using C++, 3 ed.
- Data Structures and Algorithm Analysis in C++ by Mark Allen Weiss fourth edition: Chapter 3 (3.1-3.5)
- Data Structures and Algorithms in C++ by Michael T. Goodrich, Roberto Tamassia, David M. Mount Second Edition : Chapter 3 (3.1-3.2)